ASSIGNMENT 9

Advanced SQL: Joins and Subqueries

Project: Airline Reservation System | Course: Database Management Systems

Objectives covered

- Advanced queries using joins and subqueries
- Meaningful reports for the airline system
- Multi-table reports, summary statistics and related records
- SQL, output and explanation for each report

1. Introduction

Real reports need data from several tables at once. This assignment builds twenty reports (R1 to R20) with **INNER, LEFT, RIGHT and self joins**, **scalar, IN, EXISTS and correlated subqueries**, a derived table, a common table expression (CTE) and UNION. All outputs are the real results on the sample data of Assignment 7 (SQLite 3.45).

Join / subquery type	Used for	Reports
INNER JOIN	Rows that match in both tables (the usual relationship traversal)	R1–R5
LEFT / RIGHT JOIN	Keep rows of one table even without a match (find 'missing' data)	R6–R8
SELF JOIN	Compare rows of the same table (connecting flights, same-route flights)	R9, R10
Scalar / IN subquery	Compare with an aggregate or a list computed by an inner query	R11–R13
EXISTS / NOT EXISTS	Test for the presence of related rows	R14, R15
Correlated subquery	Inner query uses a value of the current outer row	R16, R17
Derived table, CTE, UNION	Multi-step statistics and combined result sets	R18–R20

2. INNER JOIN Reports

R1 Flight schedule with airline, aircraft and route names (5-table join)

```sql
SELECT f.flight_number, al.airline_name, ac.model, o.city AS origin, d.city AS destination,
       f.departure_time, f.status
FROM flight f
JOIN aircraft ac ON ac.aircraft_id = f.aircraft_id
JOIN airline  al ON al.airline_id  = ac.airline_id
JOIN airport  o  ON o.airport_code = f.origin_code
JOIN airport  d  ON d.airport_code = f.destination_code
WHERE f.status IN ('Scheduled','Delayed')
ORDER BY f.departure_time
LIMIT 12
```

OUTPUT

flight_number	airline_name	model	origin	destination	departure_time	status
6E-501	IndiGo	Airbus A320neo	Bengaluru	New Delhi	2026-11-02 06:00:00	Scheduled
AI-807	Air India	Airbus A320	Bengaluru	Mumbai	2026-11-02 07:30:00	Scheduled
6E-502	IndiGo	Airbus A320neo	New Delhi	Bengaluru	2026-11-02 10:00:00	Scheduled
AI-808	Air India	Airbus A320	Mumbai	Bengaluru	2026-11-02 11:00:00	Delayed
QP-1305	Akasa Air	Boeing 737-8 MAX	Bengaluru	Hyderabad	2026-11-03 08:00:00	Scheduled
SG-612	SpiceJet	Boeing 737-800	New Delhi	Mumbai	2026-11-03 09:15:00	Scheduled
6E-231	IndiGo	Airbus A320neo	Bengaluru	Chennai	2026-11-03 18:30:00	Scheduled
SQ-503	Singapore Airlines	Airbus A350-900	Chennai	Singapore	2026-11-03 22:10:00	Scheduled
EK-565	Emirates	Boeing 777-300ER	Bengaluru	Dubai	2026-11-04 03:50:00	Scheduled
AI-505	Air India	Airbus A321	New Delhi	Kolkata	2026-11-04 14:00:00	Scheduled
AI-111	Air India	Boeing 787-8 Dreamliner	New Delhi	London	2026-11-05 02:00:00	Scheduled
SG-148	SpiceJet	Boeing 737-800	Mumbai	Hyderabad	2026-11-06 12:45:00	Scheduled

12 rows in set

Explanation: The flight table stores only codes and ids. Joining aircraft→airline and airport (twice, once as origin and once as destination, using aliases o and d) turns them into readable names. This is the passenger-facing schedule.

R2 Passenger itinerary: every active ticket with booking reference

```sql
SELECT p.first_name || ' ' || p.last_name AS passenger, b.booking_ref, f.flight_number,
       f.origin_code || '-' || f.destination_code AS route, t.seat_no, t.travel_class, t.fare
FROM ticket t
JOIN passenger p ON p.passenger_id = t.passenger_id
JOIN booking   b ON b.booking_id   = t.booking_id
JOIN flight    f ON f.flight_id    = t.flight_id
WHERE t.status <> 'Cancelled' AND f.departure_time >= '2026-11-02'
ORDER BY f.departure_time, t.seat_no
LIMIT 15
```

```
OUTPUT
+-----------------+-------------+---------------+---------+---------+-----------------+-------+
| passenger       | booking_ref | flight_number | route   | seat_no | travel_class    | fare  |
+-----------------+-------------+---------------+---------+---------+-----------------+-------+
| Vikram Singh    | YVB5JY      | 6E-501        | BLR-DEL | 2A      | Business        | 16020 |
| Rohan Mehta     | BXNDHL      | 6E-501        | BLR-DEL | 8A      | Economy         |  4680 |
| Ananya Iyer     | BXNDHL      | 6E-501        | BLR-DEL | 8B      | Economy         |  4680 |
| Mohammed Faizan | BBT84S      | AI-807        | BLR-BOM | 10A     | Economy         |  5160 |
| Kavya Reddy     | R4QJMB      | 6E-502        | DEL-BLR | 15D     | Economy         |  5100 |
| Arjun Reddy     | R4QJMB      | 6E-502        | DEL-BLR | 15E     | Economy         |  5100 |
| Sneha Reddy     | R4QJMB      | 6E-502        | DEL-BLR | 15F     | Economy         |  5100 |
| Divya Krishnan  | FMCSXW      | AI-808        | BOM-BLR | 6C      | Premium Economy |  5670 |
| Rahul Verma     | R8DQK3      | SG-612        | DEL-BOM | 20A     | Economy         |  3900 |
| Neha Verma      | R8DQK3      | SG-612        | DEL-BOM | 20B     | Economy         |  3900 |
| Lakshmi Menon   | VNL4K9      | 6E-231        | BLR-MAA | 11D     | Economy         |  3000 |
| Gautham Menon   | VNL4K9      | 6E-231        | BLR-MAA | 11E     | Economy         |  3000 |
| David Thomas    | K2XSVC      | SQ-503        | MAA-SIN | 30A     | Economy         | 24800 |
| Emily Thomas    | K2XSVC      | SQ-503        | MAA-SIN | 30B     | Economy         | 24800 |
| Sanjay Gupta    | BPVC32      | EK-565        | BLR-DXB | 3A      | Business        | 54180 |
+-----------------+-------------+---------------+---------+---------+-----------------+-------+
15 rows in set
```

Explanation: Four tables are joined through the ticket association table to print boarding-list style rows for upcoming flights. || concatenates text (MySQL: CONCAT).

R3 Booking summary: who booked, how many tickets, amount and payment status

```sql
SELECT b.booking_ref, bk.first_name || ' ' || bk.last_name AS booked_by, b.status AS booking_status,
       (SELECT COUNT(*) FROM ticket t WHERE t.booking_id = b.booking_id) AS tickets,
       b.total_amount,
       (SELECT COALESCE(SUM(pay.amount), 0) FROM payment pay
        WHERE pay.booking_id = b.booking_id AND pay.status = 'Success') AS paid
FROM booking b
JOIN passenger bk ON bk.passenger_id = b.passenger_id
ORDER BY b.total_amount DESC
LIMIT 10
```

OUTPUT

booking_ref	booked_by	booking_status	tickets	total_amount	paid
UYJ2NA	Karthik Subramanian	Confirmed	1	145600	145600
BPVC32	Sanjay Gupta	Confirmed	2	108360	108360
JGW9GE	Priya Nair	Confirmed	2	53520	53520
K2XSVC	David Thomas	Confirmed	2	49600	49600
LW5XW9	Kavya Reddy	Cancelled	1	21000	21000
YVB5JY	Vikram Singh	Confirmed	1	16020	16020
R4QJMB	Sneha Reddy	Confirmed	3	15300	15300
BXNDHL	Rohan Mehta	Confirmed	2	9360	9360
D5BUXU	Priya Nair	Confirmed	2	9020	9020
YFEF9V	Ishita Bose	Confirmed	1	8100	8100

10 rows in set

Explanation: Joins booking to the booker and uses scalar subqueries for the two child tables. Joining both ticket and payment directly would multiply rows (a booking with 2 tickets and 1 payment would count that payment twice), so each child table is summarised separately.

R4 Revenue by airline (active tickets)

```sql
SELECT al.airline_name, COUNT(*) AS tickets_sold, SUM(t.fare) AS revenue, ROUND(AVG(t.fare), 2) AS avg_fare
FROM ticket t
JOIN flight   f  ON f.flight_id    = t.flight_id
JOIN aircraft ac ON ac.aircraft_id = f.aircraft_id
JOIN airline  al ON al.airline_id  = ac.airline_id
WHERE t.status <> 'Cancelled'
GROUP BY al.airline_name
ORDER BY revenue DESC
```

OUTPUT

airline_name	tickets_sold	revenue	avg_fare
Air India	7	179760	25680
Emirates	4	161880	40470
IndiGo	11	58800	5345.45
Singapore Airlines	2	49600	24800
SpiceJet	3	11760	3920
Akasa Air	1	4100	4100

6 rows in set

Explanation: A management report: the airline is reached through ticket → flight → aircraft → airline, as the design stores the airline only on the aircraft (3NF).

R5 Seat occupancy (load factor) of upcoming flights

```sql
SELECT f.flight_number, f.departure_time, ac.total_seats, COUNT(t.ticket_id) AS seats_sold,
       ROUND(100.0 * COUNT(t.ticket_id) / ac.total_seats, 2) AS load_factor_pct
FROM flight f
JOIN aircraft ac ON ac.aircraft_id = f.aircraft_id
JOIN ticket t    ON t.flight_id    = f.flight_id AND t.status <> 'Cancelled'
WHERE f.status IN ('Scheduled','Delayed')
GROUP BY f.flight_id, f.flight_number, f.departure_time, ac.total_seats
ORDER BY load_factor_pct DESC
LIMIT 8
```

```
OUTPUT
+---------------+---------------------+-------------+------------+-----------------+
| flight_number | departure_time      | total_seats | seats_sold | load_factor_pct |
+---------------+---------------------+-------------+------------+-----------------+
| 6E-501        | 2026-11-02 06:00:00 |         186 |          3 |            1.61 |
| 6E-502        | 2026-11-02 10:00:00 |         186 |          3 |            1.61 |
| 6E-231        | 2026-11-03 18:30:00 |         186 |          2 |            1.08 |
| SG-612        | 2026-11-03 09:15:00 |         189 |          2 |            1.06 |
| SQ-503        | 2026-11-03 22:10:00 |         253 |          2 |            0.79 |
| AI-807        | 2026-11-02 07:30:00 |         168 |          1 |            0.60 |
| AI-808        | 2026-11-02 11:00:00 |         168 |          1 |            0.60 |
| AI-509        | 2026-11-09 19:00:00 |         168 |          1 |            0.60 |
+---------------+---------------------+-------------+------------+-----------------+
8 rows in set
```

Explanation: Seats sold divided by aircraft capacity, as a percentage. Low values reflect the small sample data; in practice this report drives pricing and marketing decisions.

3. OUTER JOIN Reports (finding missing relationships)

R6 Airports with no departing flights (LEFT JOIN)

```sql
SELECT a.airport_code, a.city, a.country
FROM airport a
LEFT JOIN flight f ON f.origin_code = a.airport_code
WHERE f.flight_id IS NULL
```

```
OUTPUT
+--------------+-----------+----------------+
| airport_code | city      | country        |
+--------------+-----------+----------------+
| CCU          | Kolkata   | India          |
| COK          | Kochi     | India          |
| GOI          | Goa       | India          |
| SIN          | Singapore | Singapore      |
| LHR          | London    | United Kingdom |
+--------------+-----------+----------------+
5 rows in set
```

Explanation: LEFT JOIN keeps every airport; for airports without flights the flight columns are NULL, and WHERE ... IS NULL isolates them. Goa (GOI) has no flights at all; Kochi, Singapore, London and Kolkata appear only as destinations, so they have no departures.

R7 Same result with RIGHT JOIN, plus aircraft that never fly

```sql
SELECT ac.registration_no, ac.model, ac.status, al.airline_name
FROM flight f
RIGHT JOIN aircraft ac ON ac.aircraft_id = f.aircraft_id
JOIN airline al        ON al.airline_id  = ac.airline_id
WHERE f.flight_id IS NULL
```

```
OUTPUT
+-----------------+-----------------+-------------+--------------+
| registration_no | model           | status      | airline_name |
+-----------------+-----------------+-------------+--------------+
| VT-SPQ          | Bombardier Q400 | Maintenance | SpiceJet     |
+-----------------+-----------------+-------------+--------------+
1 row in set
```

Explanation: RIGHT JOIN keeps every row of the right-hand table (aircraft). It is the mirror image of LEFT JOIN. The aircraft in maintenance has no flights. (RIGHT JOIN needs SQLite 3.39+, MySQL and PostgreSQL support it.)

R8 Passengers and how many tickets they hold, including those with none

```sql
SELECT p.passenger_id, p.first_name, p.last_name, COUNT(t.ticket_id) AS tickets
FROM passenger p
LEFT JOIN ticket t ON t.passenger_id = p.passenger_id
GROUP BY p.passenger_id, p.first_name, p.last_name
HAVING COUNT(t.ticket_id) <= 1
ORDER BY tickets, p.passenger_id
LIMIT 8
```

```
OUTPUT
+--------------+------------+-----------+---------+
| passenger_id | first_name | last_name | tickets |
+--------------+------------+-----------+---------+
|           23 | Amit       | Patel     |       0 |
|           25 | Tenzin     | Dorjee    |       0 |
|            5 | Vikram     | Singh     |       1 |
|            6 | Sneha      | Reddy     |       1 |
|            7 | Arjun      | Reddy     |       1 |
|            9 | Mohammed   | Faizan    |       1 |
|           10 | Divya      | Krishnan  |       1 |
|           11 | Rahul      | Verma     |       1 |
+--------------+------------+-----------+---------+
8 rows in set
```

Explanation: COUNT(t.ticket_id) counts only matched tickets, so unmatched passengers show 0 (COUNT(*) would wrongly show 1). Marketing can target customers with no or only one trip.

4. Self Join Reports

R9 Connecting itineraries on the same day (flights joined to flights)

```sql
SELECT f1.flight_number AS first_leg, f1.origin_code || '->' || f1.destination_code AS leg1,
       f1.arrival_time AS arrives, f2.flight_number AS second_leg,
       f2.origin_code || '->' || f2.destination_code AS leg2, f2.departure_time AS departs
FROM flight f1
JOIN flight f2 ON f2.origin_code = f1.destination_code
              AND f2.departure_time > f1.arrival_time
              AND DATE(f2.departure_time) = DATE(f1.arrival_time)
              AND f2.destination_code <> f1.origin_code
WHERE f1.status <> 'Cancelled' AND f2.status <> 'Cancelled'
```

```
OUTPUT
+-----------+----------+---------------------+------------+----------+---------------------+
| first_leg | leg1     | arrives             | second_leg | leg2     | departs             |
+-----------+----------+---------------------+------------+----------+---------------------+
| 6E-231    | BLR->MAA | 2026-11-03 19:35:00 | SQ-503     | MAA->SIN | 2026-11-03 22:10:00 |
| SG-148    | BOM->HYD | 2026-11-06 14:15:00 | QP-1410    | HYD->DEL | 2026-11-06 17:30:00 |
| EK-567    | DXB->BLR | 2026-11-08 14:00:00 | 6E-319     | BLR->COK | 2026-11-08 16:30:00 |
+-----------+----------+---------------------+------------+----------+---------------------+
3 rows in set
```

Explanation: The flight table is joined to itself: the second flight must leave from where the first one lands, later the same day, and must not simply return to the origin. This finds one-stop routes such as BLR to SIN via MAA.

R10 Different flights on the same route (self join on route)

```sql
SELECT a.flight_number AS flight_a, b.flight_number AS flight_b,
       a.origin_code || '->' || a.destination_code AS route, a.base_fare AS fare_a, b.base_fare AS fare_b
FROM flight a
JOIN flight b ON a.origin_code = b.origin_code AND a.destination_code = b.destination_code
             AND a.flight_id < b.flight_id
ORDER BY route
```

```
OUTPUT
+----------+----------+----------+--------+--------+
| flight_a | flight_b | route    | fare_a | fare_b |
+----------+----------+----------+--------+--------+
| AI-807   | AI-807   | BLR->BOM |   4300 |   4100 |
| 6E-501   | 6E-501   | BLR->DEL |   5200 |   5000 |
| EK-565   | EK-565   | BLR->DXB |  21500 |  21000 |
| AI-808   | 6E-714   | BOM->BLR |   4200 |   4000 |
| 6E-502   | AI-509   | DEL->BLR |   5100 |   5400 |
+----------+----------+----------+--------+--------+
5 rows in set
```

Explanation: The condition a.flight_id < b.flight_id lists each pair once and never pairs a flight with itself. Useful for comparing fares and times on a route.

5. Subquery Reports

R11 Flights priced above the average base fare (scalar subquery)

```sql
SELECT flight_number, origin_code, destination_code, base_fare
FROM flight
WHERE base_fare > (SELECT AVG(base_fare) FROM flight)
ORDER BY base_fare DESC
```

```
OUTPUT
+---------------+-------------+------------------+-----------+
| flight_number | origin_code | destination_code | base_fare |
+---------------+-------------+------------------+-----------+
| AI-111        | DEL         | LHR              |     52000 |
| SQ-503        | MAA         | SIN              |     24800 |
| EK-567        | DXB         | BLR              |     22300 |
| EK-565        | BLR         | DXB              |     21500 |
| EK-565        | BLR         | DXB              |     21000 |
+---------------+-------------+------------------+-----------+
5 rows in set
```

Explanation: The inner query returns one value (the average base fare, about 10,165). The outer query compares each flight with it. These are mainly the long-haul flights.

R12 Passengers who travel to international destinations (IN subquery)

```sql
SELECT DISTINCT p.passenger_id, p.first_name, p.last_name
FROM passenger p
WHERE p.passenger_id IN (SELECT t.passenger_id
                         FROM ticket t
                         JOIN flight f  ON f.flight_id = t.flight_id
                         JOIN airport d ON d.airport_code = f.destination_code
                         WHERE d.country <> 'India' AND t.status <> 'Cancelled')
ORDER BY p.passenger_id
```

```
OUTPUT
+--------------+------------+-------------+
| passenger_id | first_name | last_name   |
+--------------+------------+-------------+
|           17 | Sanjay     | Gupta       |
|           18 | Meera      | Gupta       |
|           19 | David      | Thomas      |
|           20 | Emily      | Thomas      |
|           21 | Karthik    | Subramanian |
+--------------+------------+-------------+
5 rows in set
```

Explanation: The inner query builds a list of passenger ids with a ticket to a non-Indian airport; the outer query returns their details. Useful for visa and passport-validity reminders.

R13 Flights busier than the average number of tickets per flight (subquery in HAVING)

```sql
SELECT f.flight_number, COUNT(t.ticket_id) AS tickets
FROM flight f JOIN ticket t ON t.flight_id = f.flight_id AND t.status <> 'Cancelled'
GROUP BY f.flight_id, f.flight_number
HAVING COUNT(t.ticket_id) > (SELECT COUNT(*) * 1.0 / COUNT(DISTINCT flight_id) FROM ticket WHERE status <> 'Cancelled')
```

```
+---------------+---------+
| flight_number | tickets |
+---------------+---------+
| 6E-501        |       3 |
| 6E-502        |       3 |
| SG-612        |       2 |
| 6E-231        |       2 |
| EK-565        |       2 |
| SQ-503        |       2 |
| EK-567        |       2 |
| AI-807        |       2 |
+---------------+---------+
8 rows in set
```

Explanation: A subquery computes the average tickets per flight; HAVING keeps only flights busier than that. Flights with only one or two tickets are filtered out.

R14 Bookings that had a failed payment attempt (EXISTS)

```sql
SELECT b.booking_ref, b.status, b.total_amount
FROM booking b
WHERE EXISTS (SELECT 1 FROM payment p WHERE p.booking_id = b.booking_id AND p.status = 'Failed')
```

```
+-------------+---------+--------------+
| booking_ref | status  | total_amount |
+-------------+---------+--------------+
| LBMWKL      | Pending |         3600 |
+-------------+---------+--------------+
1 row in set
```

Explanation: EXISTS is true as soon as one matching payment row is found; it is efficient because the inner query can stop early. The pending booking had two failed attempts.

R15 Passengers who have never booked or travelled (NOT EXISTS)

```sql
SELECT p.passenger_id, p.first_name, p.last_name, p.registered_on
FROM passenger p
WHERE NOT EXISTS (SELECT 1 FROM ticket t WHERE t.passenger_id = p.passenger_id)
  AND NOT EXISTS (SELECT 1 FROM booking b WHERE b.passenger_id = p.passenger_id)
```

```
+--------------+------------+-----------+---------------+
| passenger_id | first_name | last_name | registered_on |
+--------------+------------+-----------+---------------+
|           23 | Amit       | Patel     | 2024-06-14    |
|           25 | Tenzin     | Dorjee    | 2025-08-31    |
+--------------+------------+-----------+---------------+
2 rows in set
```

Explanation: NOT EXISTS finds passengers with no related rows in either table, i.e. registered customers who never bought anything.

R16 Each passenger's most expensive active ticket (correlated subquery)

```sql
SELECT p.first_name, p.last_name, t.flight_id, t.fare
FROM ticket t
JOIN passenger p ON p.passenger_id = t.passenger_id
WHERE t.status <> 'Cancelled'
  AND t.fare = (SELECT MAX(t2.fare) FROM ticket t2
                WHERE t2.passenger_id = t.passenger_id AND t2.status <> 'Cancelled')
ORDER BY t.fare DESC
LIMIT 8
```

```
OUTPUT
+------------+-------------+-----------+--------+
| first_name | last_name   | flight_id | fare   |
+------------+-------------+-----------+--------+
| Karthik    | Subramanian |        11 | 145600 |
| Sanjay     | Gupta       |         9 |  54180 |
| Meera      | Gupta       |         9 |  54180 |
| Priya      | Nair        |        15 |  26760 |
| Rohan      | Mehta       |        15 |  26760 |
| David      | Thomas      |        10 |  24800 |
| Emily      | Thomas      |        10 |  24800 |
| Vikram     | Singh       |         1 |  16020 |
+------------+-------------+-----------+--------+
8 rows in set
```

Explanation: The inner query refers to t.passenger_id of the current outer row, so it is re-evaluated for each ticket: 'is this the highest fare of this passenger?'

R17 Latest booking of each passenger who booked (correlated subquery)

```sql
SELECT b.passenger_id, p.first_name, b.booking_ref, b.booking_date, b.total_amount
FROM booking b JOIN passenger p ON p.passenger_id = b.passenger_id
WHERE b.booking_date = (SELECT MAX(b2.booking_date) FROM booking b2 WHERE b2.passenger_id = b.passenger_id)
ORDER BY b.booking_date DESC
LIMIT 8
```

```
OUTPUT
+--------------+------------+-------------+---------------------+--------------+
| passenger_id | first_name | booking_ref | booking_date        | total_amount |
+--------------+------------+-------------+---------------------+--------------+
|            8 | Kavya      | LW5XW9      | 2026-10-23 19:20:00 |        21000 |
|           24 | Ishita     | YFEF9V      | 2026-10-23 15:48:00 |         8100 |
|            2 | Priya      | JGW9GE      | 2026-10-22 19:16:00 |        53520 |
|            4 | Ananya     | HY89HR      | 2026-10-22 18:43:00 |         3520 |
|            1 | Aarav      | AZEZVP      | 2026-10-21 11:38:00 |         8060 |
|           22 | Farah      | LBMWKL      | 2026-10-21 07:56:00 |         3600 |
|           21 | Karthik    | UYJ2NA      | 2026-10-20 22:27:00 |       145600 |
|           19 | David      | K2XSVC      | 2026-10-19 13:10:00 |        49600 |
+--------------+------------+-------------+---------------------+--------------+
8 rows in set
```

Explanation: Returns only the most recent booking per customer, a typical 'last activity' report for customer service.

6. Derived Tables, CTEs and UNION

R18 Bookings paid in full, using a pre-aggregated derived table

```sql
SELECT b.booking_ref, b.total_amount, pay.paid, b.total_amount - pay.paid AS refunded_or_due
FROM booking b
JOIN (SELECT booking_id, SUM(amount) AS paid
      FROM payment WHERE status = 'Success' GROUP BY booking_id) pay
  ON pay.booking_id = b.booking_id
WHERE b.status = 'Cancelled' OR pay.paid > 50000
ORDER BY pay.paid DESC
```

```
OUTPUT
+-------------+--------------+--------+-----------------+
| booking_ref | total_amount | paid   | refunded_or_due |
+-------------+--------------+--------+-----------------+
| UYJ2NA      |       145600 | 145600 |               0 |
| BPVC32      |       108360 | 108360 |               0 |
| JGW9GE      |        53520 |  53520 |               0 |
| LW5XW9      |        21000 |  21000 |               0 |
| TXNZY7      |         3080 |   3080 |               0 |
+-------------+--------------+--------+-----------------+
5 rows in set
```

Explanation: The subquery in the FROM clause (derived table) totals the successful payments per booking first, which avoids the row-multiplication problem of joining two child tables. The outer query then lists large or cancelled bookings.

R19 Top-spending passengers compared with the average (CTE)

```sql
WITH spend AS (
    SELECT p.passenger_id, p.first_name || ' ' || p.last_name AS passenger, SUM(t.fare) AS total_spent
    FROM passenger p JOIN ticket t ON t.passenger_id = p.passenger_id
    WHERE t.status <> 'Cancelled'
    GROUP BY p.passenger_id, p.first_name, p.last_name )
SELECT passenger, total_spent,
       ROUND((SELECT AVG(total_spent) FROM spend), 2) AS average_spend
FROM spend
WHERE total_spent > (SELECT AVG(total_spent) FROM spend)
ORDER BY total_spent DESC
```

```
OUTPUT
+--------------------+-------------+---------------+
| passenger          | total_spent | average_spend |
+--------------------+-------------+---------------+
| Karthik Subramanian |      145600 |      21177.27 |
| Sanjay Gupta       |       54180 |      21177.27 |
| Meera Gupta        |       54180 |      21177.27 |
| Rohan Mehta        |       35950 |      21177.27 |
| Priya Nair         |       31270 |      21177.27 |
| David Thomas       |       24800 |      21177.27 |
| Emily Thomas       |       24800 |      21177.27 |
+--------------------+-------------+---------------+
7 rows in set
```

Explanation: A WITH clause (common table expression) names an intermediate result that is reused twice. It makes multi-step logic readable. Requires MySQL 8+, PostgreSQL or SQLite.

R20 Traffic per airport: departures plus arrivals (UNION ALL + derived table)

```sql
SELECT a.airport_code, a.city, SUM(x.departures) AS departures, SUM(x.arrivals) AS arrivals,
       SUM(x.departures + x.arrivals) AS total_movements
FROM (SELECT origin_code AS airport_code, 1 AS departures, 0 AS arrivals FROM flight WHERE status <> 'Cancelled'
      UNION ALL
      SELECT destination_code, 0, 1 FROM flight WHERE status <> 'Cancelled') x
JOIN airport a ON a.airport_code = x.airport_code
GROUP BY a.airport_code, a.city
ORDER BY total_movements DESC, a.airport_code
```

```
OUTPUT
+--------------+-----------+------------+----------+-----------------+
| airport_code | city      | departures | arrivals | total_movements |
+--------------+-----------+------------+----------+-----------------+
| BLR          | Bengaluru |          8 |        5 |              13 |
| DEL          | New Delhi |          5 |        3 |               8 |
| BOM          | Mumbai    |          3 |        3 |               6 |
| HYD          | Hyderabad |          1 |        2 |               3 |
| DXB          | Dubai     |          1 |        1 |               2 |
| MAA          | Chennai   |          1 |        1 |               2 |
| CCU          | Kolkata   |          0 |        1 |               1 |
| COK          | Kochi     |          0 |        1 |               1 |
| LHR          | London    |          0 |        1 |               1 |
| SIN          | Singapore |          0 |        1 |               1 |
+--------------+-----------+------------+----------+-----------------+
10 rows in set
```

Explanation: UNION ALL stacks origins and destinations into one list of movements; the outer query groups them per airport. Airports with no flights (GOI) do not appear; use a LEFT JOIN to include them.

7. Conclusion

Joins reassemble the normalized tables into meaningful reports (itineraries, schedules, revenue), outer joins expose missing relationships (unused airports and aircraft, inactive customers), self joins find connections between rows of the same table, and subqueries (scalar, IN, EXISTS, correlated, derived and CTE) express comparisons that a single SELECT cannot. Together they show that the 3NF design supports every report the airline needs without storing any fact twice.